مكتبة انوار الرافدين

انظمة تشغيل

(1)

الكورس الاول

ثالث. هندسة تقنيات

د. عمار فالح

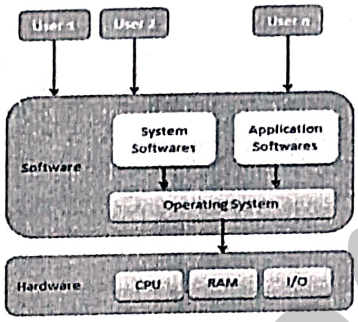
jafar walid

المكتبة الرئيسية والمعتمدة لكلية الرافدين هنا تجدون النسخة الأصلية
نتمنى لكم عام دراسياً موفقاً 2025/2026

Introduction to Operating Systems (OS)

1.1 Definition

An operating system is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs. OS acts as an intermediary between a user of a computer and the computer hardware.



1.2 Computer System

Computer system can be divided into four components:

1. **Hardware**, includes Monitor, Keyboard, CPU, Disks, Memory, etc.
2. **System Software** are the software which are more complex in nature and they are more near to computer hardware. The software is usually written in low-level languages like assembly language and includes **Operating Systems** (Microsoft Windows, Mac OS, and Linux), Compiler, and Assembler etc.
3. **Application programs** are the software which users use directly to perform different activities. These software are simple and easy to use like Browsers, Word, Excel, different Editors, Games etc. These are usually written in high-level languages, such as Python, Java and C++.
4. **Users**, whom use the overall computer system.

1.3 Operating System Architecture

In operating system Architecture, we've the following terms which defines the major components of the operating systems.

- **Kernel** – Kernel is the central component of an operating system architecture in most of the implementation. A kernel is responsible for all major operations and interaction with the hardware. A kernel manages memory, processor, input/output devices and provides interface to application programs to interact with hardware components.

Jafar-Walid

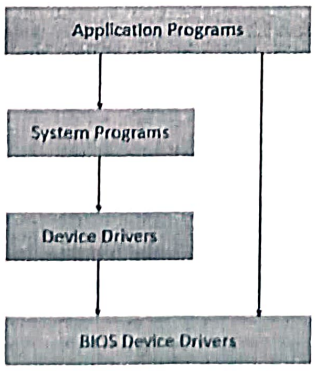
- **Shell** – Shell is an interface of an operating system. It can be command line interface or a graphical user interface. User interacts with an operating system using shell. Application programs can also use shell interface to interact with underlying operating system.
- **System Software** – System software are the programs which interact with Kernel and provides interface for security management, memory management and other low level activities.
- **Application Programs** – Application software/Programs are the one using which a user interacts with the operating system. For example, a word processor to create a document and save it on the file system, a notepad to create notes etc.

The following are various popular implementations of Operating System architectures.

- Simple Architecture.
- Monolith Architecture.
- Micro-Kernel Architecture.
- Exo-Kernel Architecture.
- Layered Architecture.
- Modular Architecture.
- Virtual Machine Architecture.

1.3.1 Simple Architecture

There are many operating systems that have a rather simple structure. These started as small systems and rapidly expanded much further than their scope. A common example of this is MS-DOS. It was designed simply for a niche amount for people. There was no indication that it would become so popular.



jafar-walid

Advantages:

- Easy Development.
- Better Performance.

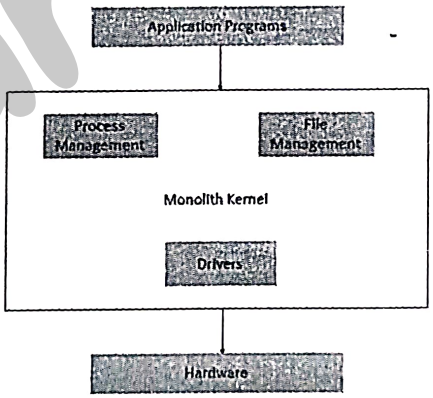
Disadvantages:

- Frequent System Failures.
- Poor Maintainability.

1.3.2 Monolith Architecture

In monolith architecture operating system, a central piece of code called kernel is responsible for all major operations of an operating system. Such operations include file management, memory management, device management and so on. The kernel is the main component of an operating system and it provides all the services of an operating system to the application programs and system programs.

The kernel has access to the all the resources and it acts as an interface with application programs and the underlying hardware. A monolithic kernel architecture promotes timesharing, multiprogramming model and was used in old banking systems.



Advantages:

- Easy Development.
- Performance.

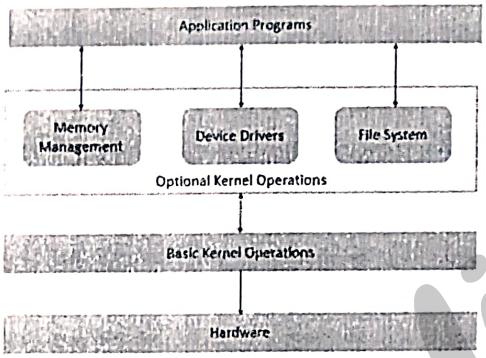
Disadvantages:

- Crash Prone.
- Difficult to enhance.

jafar_walid

1.3.3 Micro-Kernel Architecture

As in case monolith architecture, there was single kernel, in micro-kernel, we have multiple kernels each one specialized in particular service. Each microkernel is developed independent to the other one and makes system more stable. If one kernel fails, the operating system will keep working with other kernel's functionalities.



Advantages:

- Reliable and Stable.
- Maintainability.

Disadvantages:

- Complex to Design.
- Performance Degradation.

1.3.4 Exo-Kernel Architecture

Exo-Kernel Architecture operating system was designed and developed at MIT. The aim of this design was to keep Kernel size minimal while allowing the application programs to manage hardware resources directly. The purpose of removing abstraction of operating system for hardware resources was to enable application programmer to write high performance code while Exo-kernel handles other operations.

Advantages:

- High Performance.
- Application Control.

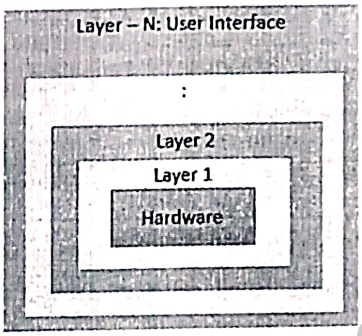
Disadvantages:

- Unreliable and Unsafe.
- Complex Design.

jafar-walid

1.3.5 Layered Architecture

One way to achieve modularity in the operating system is the layered approach. In this, the bottom layer is the hardware and the topmost layer is the user interface. As seen from the image below, each upper layer is built on the bottom layer. All the layers hide some structures, operations etc. from their upper layers.



Advantages:

- High Customizable.
- Verifiable.

Disadvantages:

- Less Performant.
- Complex designing.

jafar_walid

1.4 Two Views of OS

1- The user's view of the computer varies according to the interface being used. Many computer users sit with a laptop or in front of a PC consisting of a monitor, keyboard, and mouse. The user need:

- Ease of use.
- Good performance.

2- The system's point of view, the operating system is the program most intimately involved with the hardware. In this case the OS must deals with:

- Resource allocator (CPU, memory, storage and I/O devices), operating system must be acts as the best manager of these resources.
- Control program, OS manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with the operation and control of I/O devices.

1.5 Types and classifications of OS

They are two types of classification:

A - **Depending on platforms:**

1- **Desktop Operating Systems** (Designed for personal computers and laptops)
- Windows: A widely used OS for many PCs.
- Mac OS: The operating system for Apple's Macintosh computers.
- Linux: An open-source operating system popular in servers and workstations.

2- **Mobile Operating Systems** (Optimized for smartphones and tablets)
- Android: Google's OS for mobile devices.
- iOS: Apple's OS for iPhones and iPads.

3- **Embedded Operating Systems** (Found in specialized devices, such as smart appliances or industrial machines).

B - **According to functionality:**

1- **Batch Operating System**: Processes jobs in batches, improving efficiency.

2- **Multiprogramming Operating System**: Keeps the CPU busy by running multiple programs simultaneously, sharing them among jobs.

3- **Multitasking Operating System**: An extension of multiprogramming, allowing users to run many programs at once.

4- **Time-Sharing Operating System**: Divides CPU time among multiple users or programs, providing interactive access to computer resources.

5- **Network Operating System (NOS)**: Facilitates resource sharing and communication between computers over a network.

6- **Distributed Operating System**: Spreads tasks and resources across multiple interconnected computers rather than a single machine.

7- **Real-Time Operating System (RTOS)**: Designed for applications with strict timing deadlines, ensuring critical operations are completed on time.

jafar-walid

1.6 Evolution of OS:

Operating systems have been evolving through the years. In the 1950s, computers were limited to running one program at a time like a calculator, but later in the following decades, computers began to include more and more software programs, sometimes called libraries, that formed the basis for today's operating systems.

The first Operating System was created by General Motors in 1956 to run a single IBM mainframe computer, its name was the IBM 704. IBM was the first computer manufacturer to develop operating systems and distribute them in its computers in the 1960s.

- 0^{th} Generation (1940 - early 1950s)

  This computer in this generation were inefficient and dependent on the varying competencies of the individual programmer as operators.

- First Generation (1951-1956)

  The first generation marked the beginning of commercial computing including the introduction of Eckert and Mauchlys UNIVAC I in early 1951, and a bit later, the IBM 701.

- Second Generation (1956-1964)

  The second generation of computer hardware was most notably characterized by transistors replacing vacuum tubes as the hardware component technology. The first operating system GMOS was developed by the IBM computer.

- Third Generation (1964-1979)

  The third generation officially began in April 1964 with IBMs announcement of its System/360 family of computers. Hardware technology began to use integrated circuits (ICs) which yielded significant advantages in both speed and economy.

- Fourth Generation (1979 Present)

  The fourth generation is characterized by the appearance of the personal computer and the workstation. The component technology of the third generation, was replaced by very large scale integration (VLSI). Many Operating Systems which we are using today like Windows, Linux, Mac OS etc. developed in the fourth generation.

Jafar-Walid

1.7 Operating System Functions

Briefly, the following are some of important functions of an operating System:

- Process Management
- I/O Device Management
- File Management
- Network Management
- Main Memory Management
- Secondary Storage Management
- Security Management
- Command Interpreter System
- Control over system performance
- Job Accounting
- Error Detection and Correction
- Coordination between other software and users
- Many more other important tasks

jafar_walid

System Calls

2.1 Operating System Services

Operating systems provide an environment for execution of programs and services to programs and users:

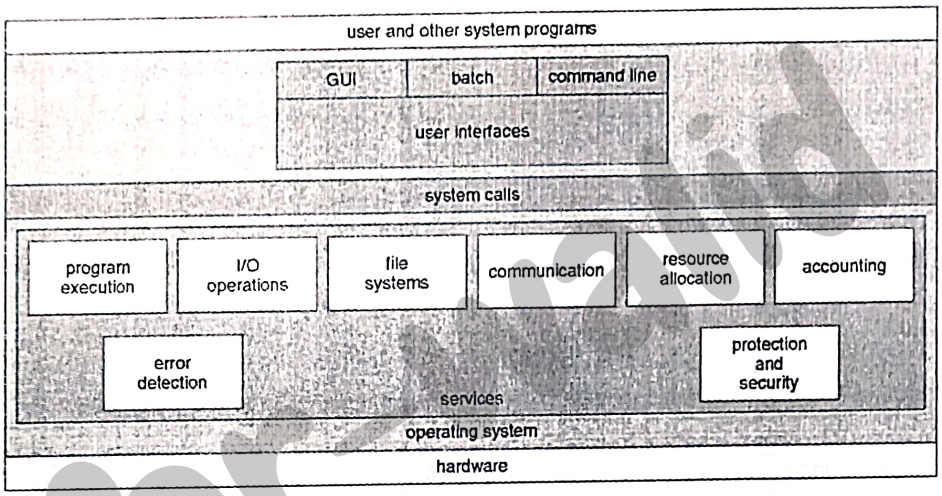
- One set of operating-system services provides functions that are helpful to the user:
  1- User interface - Almost all operating systems have a user interface (UI). Varies between Command-Line (CLI), Graphics User Interface (GUI), Batch.
  2- Program execution - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  3- I/O operations - A running program may require I/O, which may involve a file or an I/O device
- One set of operating-system services provides functions that are helpful to the user (Cont.):
  1- File-system manipulation - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
  2- Communications – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS).
  3- Error detection – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program.
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing.
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system.
- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  1- Resource allocation - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.

Jafar-Walid

2- Accounting - To keep track of which users use how much and what kinds of computer resources

3- Protection and security - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other

- Protection involves ensuring that all access to system resources is controlled
- Security of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts



1.2 User interface of OS

There are four types of user interface:

1- **CLI** or command line interpreter allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program.
- Sometimes multiple flavors implemented – shells.
- Primarily fetches a command from user and executes it.
- Sometimes commands built-in, sometimes just names of programs
  - If the latter, adding new features doesn't require shell modification.

2- **GUI** or graphical user interface, this is a User-friendly desktop metaphor interface:

- Usually mouse, keyboard, and monitor.
- Icons represent files, programs, actions, etc.
- Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a folder)

3- **Touch-Screen Interface**:

- Because either a command-line interface or a mouse-and-keyboard system is impractical for most mobile systems, smartphones and handheld tablet computers typically use a touch-screen interface.
- Users interact by making gestures on the touch screen for example, pressing and swiping fingers across the screen to do the function.

4- **Choice of Interface**:

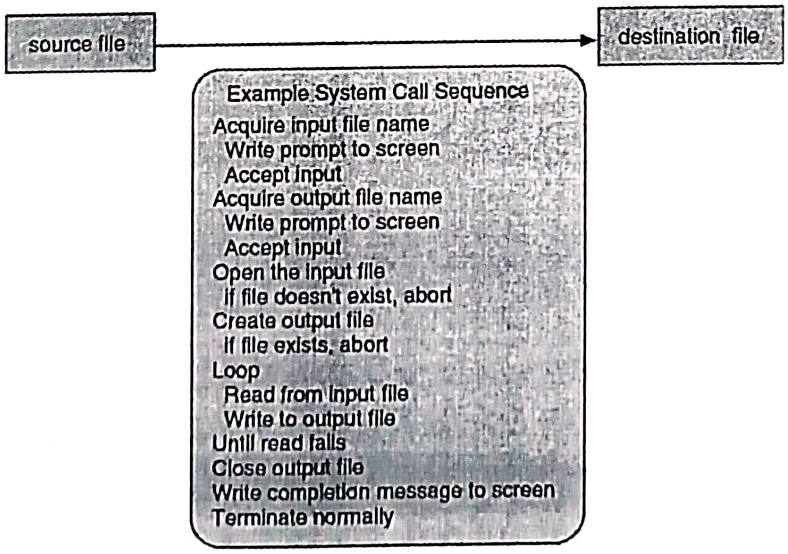
In this type OS can use both a command-line or GUI interface. Usually the person uses the GUI interface while the administrator is mostly use CLI interface.

1.3 System Calls

System calls provide an interface to the services made available by an operating system. These calls are generally available as functions written in C and C++, although certain low-level tasks (for example, tasks where hardware must be accessed directly) may have to be written using assembly-language instructions.

- Programming interface to the services provided by the OS.
- Typically written in a high-level language (C or C++).
- Mostly accessed by programs via a high-level Application Programming Interface (API) rather than direct system call use.
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM).

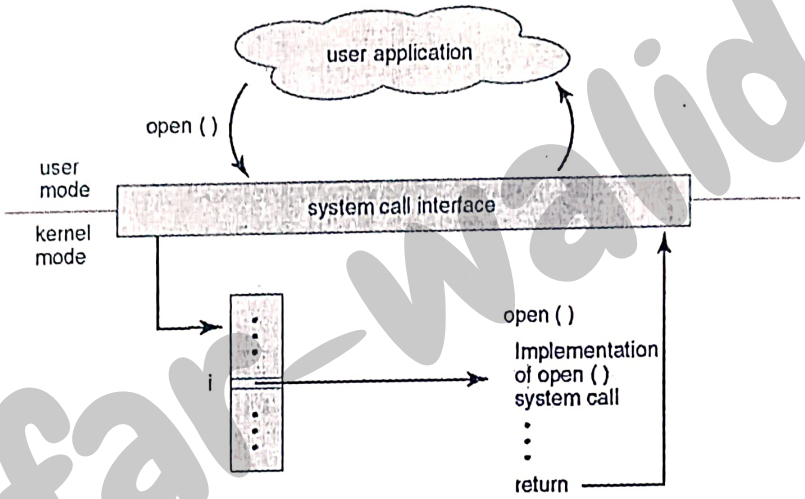
Example of system call: System call sequence to copy the contents of one file to another file, see the figure in next page.



jafar-wald

1.4 System Call Implementation

1- Typically, a number associated with each system call.
   - System-call interface maintains a table indexed according to these numbers.
2- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values.
3- The caller need know nothing about how the system call is implemented
   - Just needs to obey API and understand what OS will do as a result call.
   - Most details of OS interface hidden from programmer by API.
     - Managed by run-time support library (set of functions built into libraries included with compiler).



jafar-walid

1.5 Types of system calls

1- Process control
   - create process, terminate process.
   - end, abort.
   - load, execute.
   - get process attributes, set process attributes.
   - wait for time.
   - wait event, signal event.
   - allocate and free memory.
   - Dump memory if error.
   - **Debugger** for determining **bugs, single step** execution.
   - **Locks** for managing access to shared data between processes.
2- File management

- create file, delete file.
- open, close file.
- read, write, reposition.
- get and set file attributes.

3- Device management
- Device management.
- read, write, reposition.
- get device attributes, set device attributes.
- logically attach or detach devices.

4- Information maintenance
- Information maintenance
- get system data, set system data
- get and set process, file, or device attributes

5- Communications
- create, delete communication connection.
- send, receive messages if **message passing model** to **host name** or **process name** from **client** to **server.**
- **Shared-memory model** creates and gain access to memory regions.
- transfer status information.
- attach and detach remote devices.

6- Protection
- Control access to resources.
- Get and set permissions.
- Allow and deny user access.

jafar-walid

Solves Questions:

Q1: Answer by **True** or **False** for the following.

1- System calls provide an interface to the services made available by an operating system. (True)
2- Operating systems provide an environment for execution of programs only. (False)
3- There are four types of user interface in OS. (True)
4- Almost all operating systems have a user interface (UI). Varies between Command-Line (CLI), Graphics User Interface (GUI), Batch. (True)
5- In program execution, the OS must be able to load a program into memory and to run that program, end execution normally or abnormally. (True)

6- System calls provide an interface typically written in a high-level language (Python). (False)
7- Typically, a number associated with each system call. (True)
8- There are six types of system calls. (True)

Q2: Explain CLI interface of OS.

Sol.:

CLI or command line interpreter allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program.
- Sometimes multiple flavors implemented – shells.
- Primarily fetches a command from user and executes it.
- Sometimes commands built-in, sometimes just names of programs
  - If the latter, adding new features doesn't require shell modification.

Q3: Explain process control system call.

Sol.:

- create process, terminate process.
- end, abort.
- load, execute.
- get process attributes, set process attributes.
- wait for time.
- wait event, signal event.
- allocate and free memory.
- Dump memory if error.
- **Debugger** for determining **bugs, single step** execution.
- **Locks** for managing access to shared data between processes.

Q4: Explain protection system call.

Sol.:

- Control access to resources.
- Get and set permissions.
- Allow and deny user access.

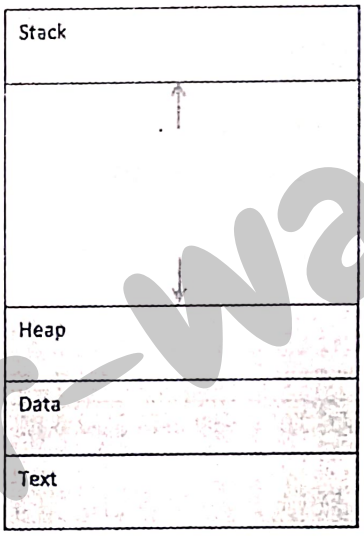
jafar-walid

Operating Systems Processes

3.1 Process

A process is basically a program in execution, is an active instance of that program currently being executed by the computer's operating system. The execution of a process must progress in a sequential fashion.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections stack, heap, text and data. The following image shows a simplified layout of a process inside main memory.



S.N.	Component & Description
1	**Stack** The process Stack contains the temporary data such as method/function parameters, return address, and local variables.
2	**Heap** This is a dynamically allocated memory to a process during its runtime.
3	**Text** This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
4	**Data** This section contains the global and static variables.

3.2 Program

A program is a piece of code which may be a single line or millions of lines written as a text file stored on a disk. A computer program is usually written by a

jafar-walid

computer programmer in a programming language. For example, here is a simple program written in Python programming language.

```
x=3
y=7
print("The sum of {} and {} is {}".format(x,y,x+y))
```

A computer program is a collection of instructions that performs a specific task when executed by a computer. When we compare a program with a process, we can conclude that a process is a dynamic instance of a computer program.

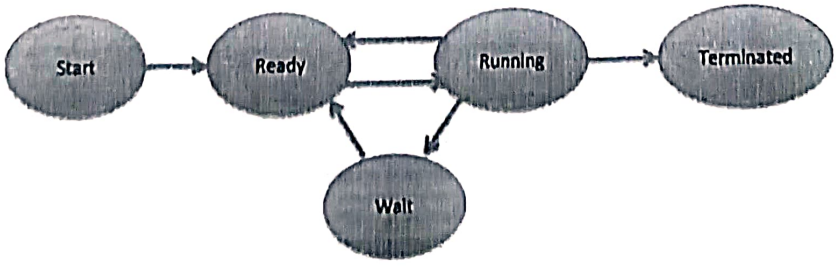
A part of a computer program that performs a well-defined task is known as an **algorithm**. A collection of computer programs, libraries and related data are referred to as a **software**.

3.3 Process Life Cycle

When a process executes, it passes through different states. These stages may differ in different operating systems, and the names of these states are also not standardized.

In general, a process can have one of the following five states at a time.

S.N.	State & Description
1	**Start** This is the initial state when a process is first started/created.
2	**Ready** The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after **Start** state or while running it by but interrupted by the scheduler to assign CPU to some other process.
3	**Running** Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.
4	**Waiting** Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.
5	**Terminated or Exit** Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.



Jasar-Walid

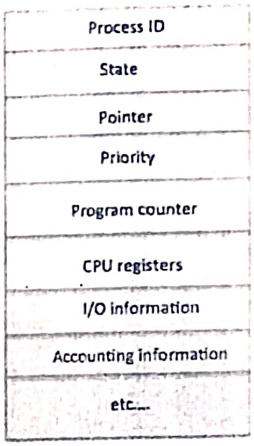
3.4 Process Control Block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process as listed below in the table.

S.N.	Information & Description
1	**Process State** The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2	**Process privileges** This is required to allow/disallow access to system resources.
3	**Process ID** Unique identification for each of the process in the operating system.
4	**Pointer** A pointer to parent process.
5	**Program Counter** Program Counter is a pointer to the address of the next instruction to be executed for this process.
6	**CPU registers** Various CPU registers where process need to be stored for execution for running state.
7	**CPU Scheduling Information** Process priority and other scheduling information which is required to schedule the process.
8	**Memory management information** This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9	**Accounting information** This includes the amount of CPU used for process execution, time limits, execution ID etc.
10	**IO status information** This includes a list of I/O devices allocated to the process.

The architecture of a PCB is completely dependent on Operating System and may contain different information in different operating systems. Here is a simplified diagram of a PCB.

jafar-walid



3.5 Process Scheduling

The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.

Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

3.6 Categories of Scheduling

There are two categories of scheduling:

1. **Non-preemptive:** Here the resource can't be taken from a process until the process completes execution. The switching of resources occurs when the running process terminates and moves to a waiting state.
2. **Preemptive:** Here the OS allocates the resources to a process for a fixed amount of time. During resource allocation, the process switches from running state to ready state or from waiting state to ready state. This switching occurs as the CPU may give priority to other processes and replace the process with higher priority with the running process.

3.7 Process Scheduling Queues

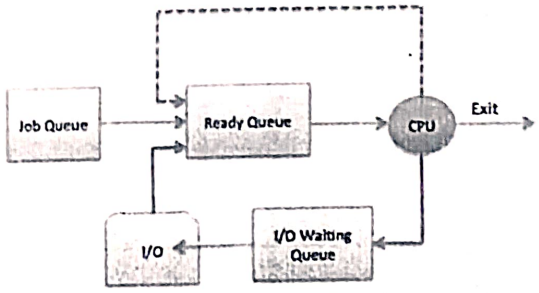
The OS maintains all Process Control Blocks (PCBs) in Process Scheduling Queues. The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.

The Operating System maintains the following important process scheduling queues:

- **Job queue** – This queue keeps all the processes in the system.

Jafar-Walid

- **Ready queue** – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- **Device queues** – The processes which are blocked due to unavailability of an I/O device constitute this queue.



The OS can use different policies to manage each queue (FIFO, Round Robin, Priority, etc.). The OS scheduler determines how to move processes between the ready and run queues which can only have one entry per processor core on the system; in the above diagram, it has been merged with the CPU. There are two-state of process model refers to running (when a process is created, it enters into the system as in the running state) and non-running (Processes that are not running are kept in queue, waiting for their turn to execute.).

3.8 Schedulers

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types :

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

3.8.1 Long Term Scheduler

It is also called a **job scheduler**. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

On some systems, the long-term scheduler may not be available or minimal. Time-sharing operating systems have no long term scheduler. When a

Jafar_Wahid

process changes the state from new to ready, then there is use of long-term scheduler.

3.8.2 Short Term Scheduler

It is also called as **CPU scheduler**. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.

Short-term schedulers, also known as dispatchers, make the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

3.8.3 Medium Term Scheduler

Medium-term scheduling is a part of **swapping**. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

3.9 Comparison among Scheduler

S.N.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

Jafar-Walid

Scheduling Algorithms

4.1 Introduction

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms. We explain the following algorithms in details.

- First-Come, First-Served (FCFS) Scheduling.
- Shortest-Job-Next (SJN) Scheduling.
- Priority Scheduling.
- Round Robin(RR) Scheduling.
- Multiple-Level Queues Scheduling.

These algorithms are either **non-preemptive or preemptive**. Non-preemptive algorithms are designed so that once a process enters the running state, it cannot be preempted until it completes its allotted time, whereas the preemptive scheduling is based on priority where a scheduler may preempt a low priority running process anytime when a high priority process enters into a ready state.

4.2 FCFS Scheduling

FCFS is considered as simplest CPU-scheduling algorithm. In FCFS algorithm, the process that requests the CPU first is allocated in the CPU first. The implementation of FCFS algorithm is managed with FIFO (First in first out) queue. FCFS scheduling is non-preemptive. Non-preemptive means, once the CPU has been allocated to a process, that process keeps the CPU until it executes a work or job or task and releases the CPU, either by requesting I/O.

Advantages of FCFS Scheduling:

- It is an easy algorithm to implement since it does not include any complex way.
- Every task should be executed simultaneously as it follows FIFO queue.
- FCFS does not give priority to any random important tasks first so its a fair scheduling.

Disadvantages of FCFS Scheduling

- FCFS results in convoy effect which means if a process with higher burst time comes first in the ready queue then the processes with lower burst time may get blocked and that processes with lower burst time may not be able to get the CPU if the higher burst time task takes time forever.
- If a process with long burst time comes in the line first then the other short burst time process has to wait for a long time, so it is not much good as time-sharing systems.
- Since it is non-preemptive, it does not release the CPU before it completes its task execution completely.

Jafar-Walid

In CPU-scheduling problems some terms are used while solving the problems, so for conceptual purpose the terms are discussed as follows –

- **Arrival time (AT)** – Arrival time is the time at which the process arrives in ready queue.
- **Burst time (BT) or CPU time of the process** – Burst time is the unit of time in which a particular process completes its execution.
- **Completion time (CT)** – Completion time is the time at which the process has been terminated.
- **Turn-around time (TAT)** – The total time from arrival time to completion time is known as turn-around time. TAT can be written as,
- **Waiting time (WT)** Waiting time is the time at which the process waits for its allocation while the previous process is in the CPU for execution. WT is written as,

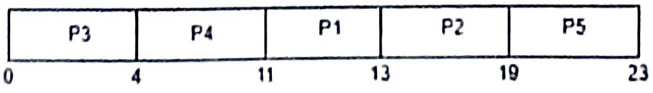
Turn-around time (TAT) = Completion time (CT) – Arrival time (AT) *or* **TAT** = Burst time (BT) + Waiting time (WT)
Waiting time (WT) = Turn-around time (TAT) – Burst time (BT)

- **Response time (RT)** – Response time is the time at which CPU has been allocated to a particular process first time.
- In case of non-preemptive scheduling, generally Waiting time and Response time is same.
- **Gantt chart** – Gantt chart is a visualization which helps to scheduling and managing particular tasks in a project. It is used while solving scheduling problems, for a concept of how the processes are being allocated in different algorithms.

Example 1: Consider the given table below and find Completion time (CT), Turn-around time (TAT), Waiting time (WT), Response time (RT), Average Turn-around time and Average Waiting time.

Process ID	Arrival time	Burst time
P1	2	2
P2	5	6
P3	0	4
P4	0	7
P5	7	4

Solution:
Gantt chart



jafar_walid

For this problem CT, TAT, WT, RT is shown in the given table.

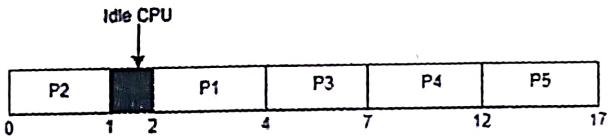
Process ID	Arrival time	Burst time	CT	TAT=CT-AT	WT=TAT-BT	RT
P1	2	2	13	13-2= 11	11-2= 9	9
P2	5	6	19	19-5= 14	14-6= 8	8
P3	0	4	4	4-0= 4	4-4= 0	0
P4	0	7	11	11-0= 11	11-7= 4	4
P5	7	4	23	23-7= 16	16-4= 12	12

Average Waiting time = (9+8+0+4+12)/5 = 33/5 = 6.6 time unit.
Average Turn-around time = (11+14+4+11+16)/5 = 56/5 = 11.2 time unit.

Example 2: Consider the given table below and find Completion time (CT), Turn-around time (TAT), Waiting time (WT), Response time (RT), Average Turn-around time and Average Waiting time.

Process ID	Arrival time	Burst time
P1	2	2
P2	0	1
P3	2	3
P4	3	5
P5	4	5

Solution:
Gantt chart



For this problem CT, TAT, WT, RT is shown in the given table.

Process ID	Arrival time	Burst time	CT	TAT=CT-AT	WT=TAT-BT	RT
P1	2	2	4	4-2= 2	2-2= 0	0
P2	0	1	1	1-0= 1	1-1= 0	0
P3	2	3	7	7-2= 5	5-3= 2	2
P4	3	5	12	12-3= 9	9-5= 4	4
P5	4	5	17	17-4= 13	13-5= 8	8

jafar-walid

Average Waiting time = (0+0+2+4+8)/5 = 14/5 = 2.8 time unit. **Average Turn-around time** = (2+1+5+9+13)/5 = 30/5 = 6 time unit.
*In idle (not-active) CPU period, no process is scheduled to be terminated so in this time it remains void for a little time.

4.3 SJF (SHORTEST JOB FIRST) Scheduling

In the Shortest Job First scheduling algorithm, the processes are scheduled in ascending order of their CPU burst times, i.e. the CPU is allocated to the process with the shortest execution time.

SJF non-preemptive scheduling:

In the non-preemptive version, once a process is assigned to the CPU, it runs into completion. Here, the short term scheduler is invoked when a process completes its execution or when a new processes arrives in an empty ready queue.

SJF preemptive scheduling:

This is the preemptive version of SJF scheduling and is also referred as Shortest Remaining Time First (SRTF) scheduling algorithm. Here, if a short process enters the ready queue while a longer process is executing, process switch occurs by which the executing process is swapped out to the ready queue while the newly arrived shorter process starts to execute. Thus the short term scheduler is invoked either when a new process arrives in the system or an existing process completes its execution.

Advantages of SJF Algorithm:

- In both preemptive and non-preemptive methods, the average waiting time is reduced substantially in SJF when compared to FCFS scheduling.
- SJF optimizes turnaround time to a considerable degree.
- If execution time of each process is estimated precisely, it promises maximum throughput.

Disadvantages of SJF Algorithm:

- In situations where there is an incoming stream of processes with short burst times, longer processes in the system may be waiting in the ready queue indefinitely leading to starvation.
- In preemptive SJF, i.e. SRTF, if all processes arrive at different times and at frequent intervals, the scheduler may be always working and consequently the processor may be more engaged in process switching than actual execution of the processes.
- Correct estimation of the burst time a process is a complicated process. Since the effectiveness of the algorithm is entirely based upon the burst times, an erroneous calculation may cause inefficient scheduling.

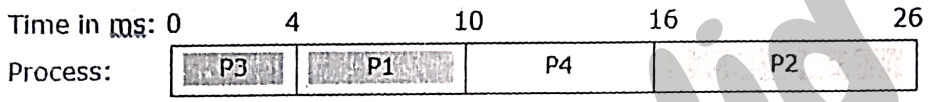
Example 1: Suppose that we have a set of four processes that have arrived at the same time in the order P1, P2, P3 and P4. The burst time in milliseconds of each process is given by the following table.

Jafar-Walid

Process	CPU Burst Time in ms
P1	6
P2	10
P3	4
P4	6

Solution:

GANTT Chart, Process P3 has the shortest burst time and so it executes first. Then we find that P1 and P4 have equal burst time of 6ms. Since P1 arrived before, CPU is allocated to P1 and then to P4. Finally, P2 executes. Thus the order of execution is P3, P1, P4, P2.



Let us compute the average turnaround time and average waiting time from the above chart.

Average Turnaround Time=Sum of Turnaround Time of each Process / Number of Processes

$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$

$= (10 + 26 + 4 + 16) / 4 = 14$ ms

Average Waiting Time= Sum of Waiting Time of Each Process / Number of processes

$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$

$= (4 + 16 + 0 + 10) / 4 = 7.5$ ms

Example 2: In the previous example, we had assumed that all the processes had arrived at the same time, a situation which is practically impossible. Here, we consider circumstance when the processes arrive at different times. Suppose we have set of four processes whose arrival times and CPU burst times are as follows.

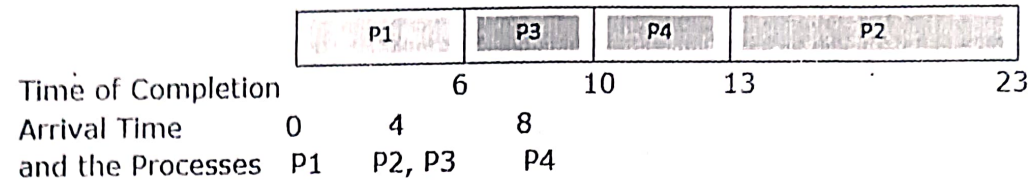
Process	Arrival Time	CPU Burst Time
P1	0	6
P2	4	10
P3	4	4
P4	8	3

Solution:

GANTT Chart, while drawing the GANTT chart, we will consider which processes have arrived in the system when the scheduler is invoked. At time 0ms,

Jafar-Walid

only P1 is there and so it is assigned to CPU. P1 completes execution at 6ms and at that time P2 and P3 have arrived, but not P4. P3 is assigned to CPU since it has the shortest burst time among current processes. P3 completes execution at time 10ms. By that time P4 has arrived and so SJF algorithm is run on the processes P2 and P4. Hence, we find that the order of execution is P1, P3, P4, P2.



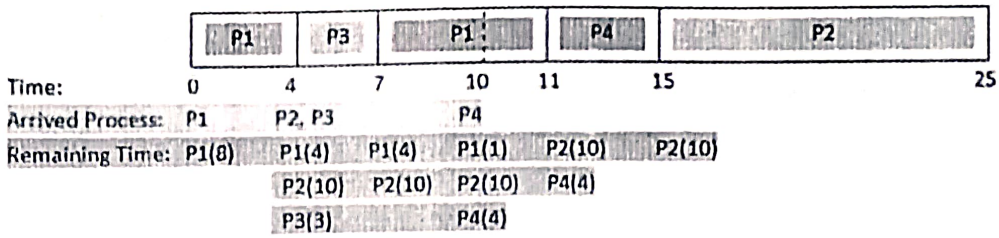
Turnaround Time of a process = Completion Time Arrival Time
$TAT_{P1} = CT_{P1} - AT_{P1} = 6 - 0 = 6$ ms
$TAT_{P2} = CT_{P2} - AT_{P2} = 23 - 4 = 19$ ms
$TAT_{P3} = CT_{P3} - AT_{P3} = 10 - 4 = 6$ ms
$TAT_{P4} = CT_{P4} - AT_{P4} = 13 - 8 = 5$ ms
Average Turnaround Time = (6 + 19+ 6 + 5) / 4 = 9 ms
Waiting Time of any process = Time of admission to CPU Arrival Time
$WT_{P1} = 0 - 0 = 0$ ms
$WT_{P2} = 13 - 4 = 9$ ms
$WT_{P3} = 6 - 4 = 2$ ms
$WT_{P4} = 10 - 8 = 2$ ms
Average Waiting Time = $(WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$
= (0 + 9 + 2 + 2) / 4 = 3.25 ms

Example 3: Preemptive SJF (SRTF) algorithm, let us now perform preemptive SJF (SRTN) scheduling on the following processes, draw GANTT chart and find the average turnaround time and average waiting time.

Process	Arrival Time	CPU Burst Time
P1	0	8
P2	4	10
P3	4	3
P4	10	4

Solution:
GANTT Chart Since this is a preemptive scheduling algorithm, the scheduler is invoked when a process arrives and when it completes execution. The scheduler computes the remaining time of completion for each of the processes in the system and selects the process having shortest remaining time left for execution.

Jafar-Walid



Initially, only P1 arrives and so it is assigned to CPU. At time 4ms, P2 and P3 arrive. The scheduler computes the remaining time of the processes P1, P2 and P3 as 4ms, 10ms and 3ms. Since, P3 has shortest time, P1 is pre-empted by P3. P3 completes execution at 7ms and the scheduler is invoked. Among the processes in the system, P1 has shortest time and so it executes. At time 10ms, P4 arrives and the scheduler again computes the remaining times left for each process. Since the remaining time of P1 is least, no process switch occurs and P1 continues to execute. In the similar fashion, the rest of the processes complete execution. From the GANTT chart, we compute the average turnaround time and the average waiting time.

Average Turnaround Time = Sum of Turnaround Time of each Process/ Number of Processes

$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$

$= ((11 - 0) + (25 - 4) + (7 - 4) + (15 - 10)) / 4 = 10$ ms

Average Waiting Time = Sum of Waiting Time of Each Process/ Number of processes

$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$

$= (3 + 11 + 0 + 1) / 4 = 3.75$ ms

4.4 Priority Scheduling:

In systems where Priority Scheduling strategies are implemented, each process is assigned a priority value. Some systems follow the scheme that lower the priority value, higher the priority; while other systems follow the scheme of higher the priority value, higher the priority. The process with the highest priority value is selected for execution.

There are two variants of Priority scheduling:

Non-preemptive Priority Scheduling:

In the non-preemptive version, once a process is assigned to the CPU, it runs into completion. Here, the scheduler is invoked when a process completes its execution or when a new process(es) arrives in an empty ready queue. The scheduler chooses the process with highest priority for execution.

Preemptive Priority Scheduling:

In the preemptive version, if a high priority process enters the system while a lower priority process is executing, process switch occurs by which the

jafar-walid

executing process is swapped out while the newly arrived higher priority process starts to execute. Thus the scheduler is invoked either when a new process arrives in the system or an existing process completes its execution.

Advantages of Priority Scheduling:

- Implementation is simple since scheduler doesnt require doing any prior calculations.
- Once CPU defines the relative relevance (priorities) of the processes, the order of execution is easily predictable.
- Higher priority processes are almost served immediately.
- Priority scheduling is particularly helpful in systems that have variety of processes each with their own needs.

Disadvantages of Priority Scheduling:

- In static priority systems, lower priority processes may need to wait indefinitely since the system is busy executing higher priority processes. This results in stagnation.
- Dynamic priority solves the stagnation problem. However, the added logic of dynamically updating priority values according to the system requires additional CPU cycles and thus increases load on the system.
- In non-preemptive priority scheduling, often a large process keeps shorter processes waiting for long time.
- In preemptive priority scheduling, a low priority process may be repeatedly pre-empted by intermittent streams of high priority processes requiring frequent context switches.

Example 1: Suppose that we have a set of four processes that have arrived at the same time in the order P1, P2, P3 and P4. The burst time in milliseconds and their priorities of the processes is given by the following table.

Process	CPU Burst Time in ms.	Priority
P1	6	2
P2	10	1
P3	4	3
P4	6	2

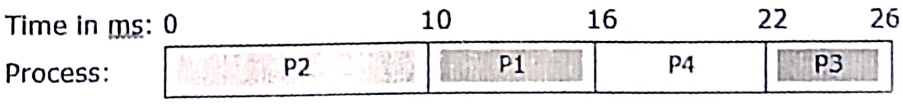
Considering that a lower priority value means higher priority, let us perform non-preemptive priority scheduling on it. We will draw the GANTT chart and find the average turnaround time and average waiting time.

Solution:

GANTT Chart, Process P2 has the highest priority and so it executes first. Then we find that both P1 and P4 have equal priority value of 2. Since P1 arrived

Jafar_Wajd

before, CPU is allocated to P1 and then to P4. Finally, P3 executes. Thus the order of execution is P2, P1, P4, P3.



Average Turnaround Time = Sum of Turnaround Time of each Process/ Number of Processes

$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$

$= (16 + 10 + 26 + 22) / 4 = 18.5$ ms

Average Waiting Time = Sum of Waiting Time of Each Process/ Number of processes

$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$

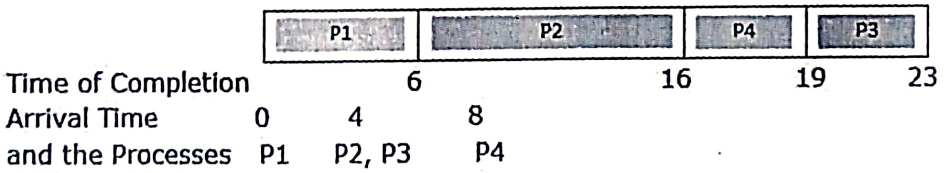
$= (10 + 0 + 22 + 16) / 4 = 10.5$ ms

Example 2: In this example, we consider a situation when the processes arrive at different times. Suppose we have set of four processes whose arrival times, CPU burst times and priorities are as follows.

Process	Arrival Time	CPU Burst Time	Priority
P1	0	6	2
P2	4	10	1
P3	4	4	2
P4	8	3	1

Solution:

GANTT Chart, at time 0ms, only P1 is there and so it is assigned to CPU. P1 completes execution at 6ms and at that time P2 and P3 have arrived. P2 has higher priority and hence assigned to CPU. P2 completes execution at time 10ms. By that time P4 has arrived having priority value 1 and so it is assigned to CPU. Once P4 completes execution, P3 is assigned to CPU. So the order of execution is P1, P2, P4, P3.



Turnaround Time of a process = Completion Time Arrival Time

$TAT_{P1} = CT_{P1} - AT_{P1} = 6 - 0 = 6$ms

$TAT_{P2} = CT_{P2} - AT_{P2} = 16 - 4 = 12$ms

$TAT_{P3} = CT_{P3} - AT_{P3} = 23 - 4 = 19$ms

$TAT_{P4} = CT_{P4} - AT_{P4} = 19 - 8 = 11$ms

jafar-walid

Average Turnaround Time = Sum of Turnaround Time of each Process / Number of Processes
= (6 + 12+ 19 + 11) / 4 = 12 ms

Waiting Time of any process = Time of admission to CPU Arrival Time

WT_{P1} = 0 - 0 = 0ms

WT_{P2} = 6 - 4 = 2ms

WT_{P3} = 19 - 4 = 15ms

WT_{P4} = 16 - 8 = 8ms

Average Waiting Time = Sum of Waiting Time of Each Process / Number of processes
= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4
= (0 + 2 +15 + 8) / 4 = 6.25 ms

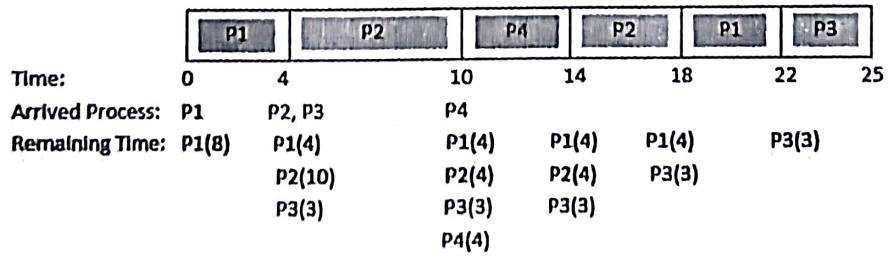
Example 3: Preemptive Priority Scheduling, in preemptive priority scheduling, if a process arrives that has higher priority than the executing process, then the higher priority process pre-empts the lower priority process. Let us consider the following set of processes whose arrival times, burst times and priorities are given in the following table.

Process	Arrival Time	CPU Burst Time	Priority
P1	0	8	3
P2	4	10	2
P3	4	3	3
P4	10	4	1

Jafar-Wajd

Solution:

GANTT Chart, at time 0ms, P1 is the only process and so it starts executing. At time 4ms, P2 and P3 arrive. Since P2 has higher priority than P1, P2 preempts P1. At 10ms, P4 arrives which has higher priority than P2 and so pre-empts P2. P4 completes execution at 14ms and leaves. The processes in the system are P1, P2 and P3, among which P2 has highest priority and so is assigned to CPU. At 18ms, P2 completes execution and so P1 and P3 are the processes in the system. Since both processes are of same priority, the scheduler selects P1 by FCFS method. When P1 completes execution, finally P3 executes. The following GANTT chart shows the order of execution.



Average Turnaround Time = Sum of Turnaround Time of each Process/ Number of Processes

$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$

$= ((22-0) + (18-4) + (25-4) + (14-10)) / 4 = 15.25$ ms

Average Waiting Time = Sum of Waiting Time of Each Process/ Number of processes

$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$

$= (14 + 4 + 18 + 0) / 4 = 7$ ms

4.5 Round Robin (RR) Scheduling:

In RR scheduling, each process gets equal time slices (or time quanta) for which it executes in the CPU in turn wise manner. When a process gets its turn, it executes for the assigned time slice and then relinquishes the CPU for the next process in queue. If the process has burst time left, then it is sent to the end of the queue. Processes enter the queue on first come first serve basis.

Round Robin scheduling is preemptive, which means that a running process can be interrupted by another process and sent to the ready queue even when it has not completed its entire execution in CPU. It is a preemptive version of First Come First Serve (FCFS) scheduling algorithm.

Working Principle of Round Robin Scheduling:

- Any new process that arrives the system is inserted at the end of the ready queue in FCFS manner.
- The first process in the queue is removed and assigned to the CPU.
- If the required burst time is less than or equal to the time quantum, the process runs to completion. The scheduler is invoked when the process completes executing to let in the next process in the ready queue to the CPU.
- If the required burst time is more than the time quantum, the process executes up to the allotted time quantum. Then its PCB (process control block) status is updated and it is added to the end of the queue. Context switch occurs and the next process in the ready queue is assigned to the CPU.
- The above steps are repeated until there are no more processes in the ready queue.

Advantages of Round Robin Scheduling:

- Round Robin scheduling is the most a fair scheduling algorithm whereby all processes are given equal time quantum for execution.
- Starvation of any process caused by indefinite waiting in ready queue is totally eliminated in RR scheduling.
- It does not require any complicated method to calculate the CPU burst time of each process prior to scheduling.

Jafar-Wala

- It is pretty simple to implement and so finds application in a wide range of situations.
- Convoy effect does not occur in RR scheduling as in First Come First Serve CPU (FCFS) scheduling.

Disadvantages of Round Robin Scheduling:

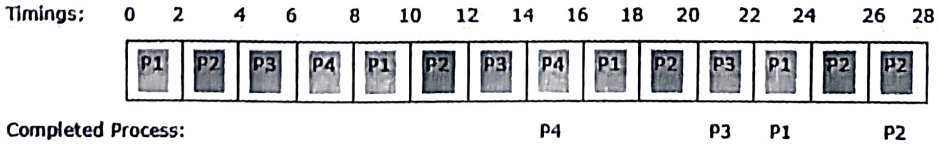
- The performance of Round Robin scheduling is highly dependent upon the chosen time quantum. This requires prudent analysis before implementation; failing which required results are not received.
- If the chosen time quantum is very large, most of the processes with complete within the burst time. In effect, RR scheduling will act as FCFS scheduling. Thus, all the limitations of FCFS will come into the system.
- If the chosen time quantum is too small, the CPU will be very busy in context switching, i.e. swapping in swapping out processes to and from the CPU and memory. This would reduce the throughput of the system since more time will be expended in context switching rather than actual execution of the processes.
- RR scheduling does not give any scope to assign priorities to processes. So, system processes which need high priority gets the same preference as background processes. This may often hamper the overall performance of a system.

Example: Let us consider a system that has four processes which have arrived at the same time in the order P1, P2, P3 and P4. The burst time in milliseconds of each process is given by the following table.

Process	CPU Burst Times in ms.
P1	8
P2	10
P3	6
P4	4

Let us consider time quantum of 2ms and perform RR scheduling on this. We will draw GANTT chart and find the average turnaround time and average waiting time.

Solution:
GANTT Chart with time quantum of 2ms.



Jafar-Walid

Average TAT = Sum of Turnaround Time of each Process / Number of Processes
=(TATP1+TATP2+TATP3+TATP4)/4
= (24 + 28 + 22+ 16) / 4 = 22.5 ms

In order to calculate the waiting time of each process, we multiply the time quantum with the number of time slices the process was waiting in the ready queue.

Average WT = Sum of Waiting Time of Each Process / Number of processes
=(WTP1+WTP2+WTP3+WTP4)/4
= (8*2 + 9*2+ 8*2+ 6*2) / 4 = 15.5 Ms.

4.6 Multilevel Queue (MLQ) Scheduling:

In multilevel queue scheduling, the processes are divided into separate categories, where category has its own distinct characteristics. In most systems, there are three types of processes, the system processes, the foreground processes and the background processes.

Advantages of Multilevel Queue Scheduling:

- The main advantage of MLQ is that it allows **setting priorities** between the processes, which is an essential requirement for practical purposes.
- MLQ enables **efficient distribution of the system resources** between the different processes.
- MLQ **reduces the response times** for high priority processes drastically. This makes the overall system more responsive.
- MLQ is highly **customizable** according to the system needs.
- MLQ improves **system performances** and increases throughput.
- It has **low scheduling overheads** in systems with varied process mix.

Limitations of Multilevel Queue Scheduling:

- A process in a low priority queue may face **undefined starvation** if there are many high priority queues each having steady streams of processes. This is the main drawback of MLQ.
- Implementation of MLQ is inherently **complex**. It requires proper design and development, failing which the entire scheduling mechanism may break down.
- Assigning fixed priorities to processes makes MLQ an **inflexible** strategy.
- The scheduling is too much dependent on the **fixed priorities**, which are assigned as soon as a process enters the system. Faulty assignment of process priorities hampers performance in big way.

Example: Let us consider as system with 3 queues having priorities 1 to 3, having processes P1 to P5 as shown in the following table.

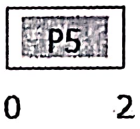
jaat-wala

Queue	Priority	Scheduling Algorithm	Processes	Arrival Time in ms	Burst Time in ms
Q1	1	RR with quanta = 2ms	P1	7	2
Q2	2	SRTN	P2	4	6
			P3	4	4
Q3	3	FCFS	P4	2	3
			P5	0	12

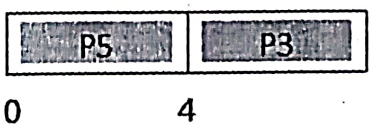
Solution:
Let us perform MLQ scheduling and draw GNATT chart of the processes.

At Time = 0ms
Event: P5 of Q3 arrives
Processes in the system: P5 in Q3.
Since, P5 is the only process, it is scheduled immediately.

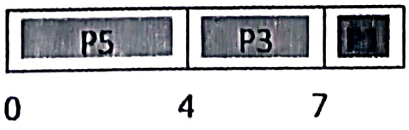
At Time = 2ms
Event: P4 of Q3 arrives
Processes in the system: P4 and P5 in Q3.
Since, both P4 and P5 are in Q3 and Q3 follows First Come First - Serve scheduling, P5 continues execution.
GANTT chart up to 2ms is



At Time = 4ms
Event: Processes P2 and P3 arrives.
Processes in the system: P2 and P3 in Q2; P4 and P5 in Q3.
Since both P2 and P3 are in Q2 while executing process P5 is in Q3, P5 is pre-empted. Among P2 and P3, P3 is assigned to CPU since Q2 follows Shortest - Remaining Time - Next scheduling
GANTT chart up to 4ms is



At Time = 7ms
Event: Processes P1 arrives.
Processes in the system: P1 in Q1; P2, P3 in Q2; P4, P5 in Q3.
Since P1 is in Q1 while executing process P3 is in Q2, P3 is pre-empted and P1 starts to execute.
GANTT chart up to 7ms is



Jafar-Walid

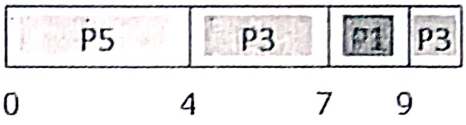
At Time = 9ms

Event: Processes P1 completes execution.

Processes in the system: P2, P3 in Q2; P4, P5 in Q3.

Since P2 and P3 are in Q2 which are at higher priority queue than P4 and P5, they compete for CPU time. P3 is assigned to CPU since Q2 uses SRTN scheduling, and remaining times of P2 and P3 are 6 and 1 respectively.

GANTT chart up to 9ms is



At Time = 10ms

Event: Processes P3 completes execution.

Processes in the system: P2 in Q2; P4, P5 in Q3.

Since P2 is in Q2 which are at higher priority queue than P4 and P5, P2 is assigned to CPU.

GANTT chart up to 10ms is

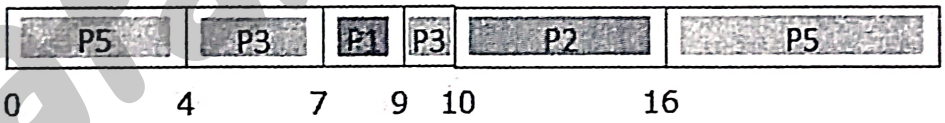
At Time = 16ms

Event: Processes P2 completes execution.

Processes in the system: P4, P5 in Q3.

Since P5 had arrived first and Q3 follows FCFS scheduling, P5 is assigned to CPU.

GANTT chart up to 16ms is



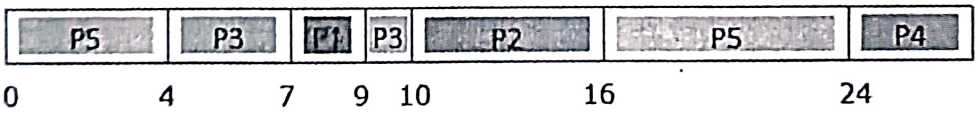
At Time = 24ms

Event: Processes P5 completes execution.

Processes in the system: P4 in Q3.

Since P4 is the only process in system, it is assigned to CPU.

GANTT chart up to 24ms is

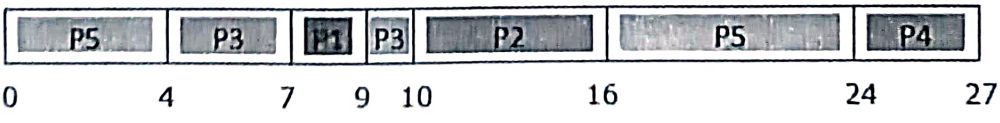


At Time = 27ms

Event: Processes P4 completes execution.

Processes in the system: none.

Final GANTT chart up to 27ms is



jafar_wald